

Модуль math в Python

Модуль **math** в Python предоставляет множество математических функций и констант, которые могут быть полезны для выполнения различных вычислений.

1. Импорт модуля math

Прежде чем использовать функции и константы из модуля **math**, его необходимо импортировать:

```
import math
```

Теперь мы можем обращаться к функциям и константам модуля через префикс **math..**

2. Основные константы

Модуль **math** предоставляет несколько полезных математических констант:

math.pi — число π (пи), приблизительно равное 3.141592653589793.

math.e — число Эйлера, приблизительно равное 2.718281828459045.

math.tau — число τ (тау), равное 2π , приблизительно 6.283185307179586.

math.inf — бесконечность.

math.nan — "не число" (Not a Number).

Пример использования констант:

```
import math
print("π =", math.pi)
print("e =", math.e)
print("τ =", math.tau)
print("Бесконечность =", math.inf)
print("Не число =", math.nan)
```

3. Основные функции

3.1. Тригонометрические функции

Модуль **math** предоставляет стандартные тригонометрические функции, такие как **sin**, **cos**, **tan**, а также их обратные функции **asin**, **acos**, **atan**.

math.sin(x) — синус угла x (в радианах).

math.cos(x) — косинус угла x (в радианах).

math.tan(x) — тангенс угла x (в радианах).

math.asin(x) — арксинус числа x .

math.acos(x) — арккосинус числа x .

math.atan(x) — арктангенс числа x .

Пример:

```
import math
angle = math.pi / 4 # 45 градусов в радианах
print("sin(45°) =", math.sin(angle))
print("cos(45°) =", math.cos(angle))
print("tan(45°) =", math.tan(angle))
```

3.2. Гиперболические функции

Модуль **math** также предоставляет гиперболические функции:

math.sinh(x) — гиперболический синус.

math.cosh(x) — гиперболический косинус.

math.tanh(x) — гиперболический тангенс.

Пример:

```
import math
x = 1.0
print("sinh(1.0) =", math.sinh(x))
print("cosh(1.0) =", math.cosh(x))
print("tanh(1.0) =", math.tanh(x))
```

3.3. Экспоненциальные и логарифмические функции

math.exp(x) — экспонента числа x (e^x).

math.log(x[, base]) — натуральный логарифм числа x (по умолчанию) или логарифм по основанию $base$.

math.log10(x) — десятичный логарифм числа x .

math.log2(x) — логарифм числа x по основанию 2.

Пример:

```
import math
x = 2.0
print("e^2 =", math.exp(x))
print("ln(2) =", math.log(x))
print("log10(100) =", math.log10(100))
print("log2(8) =", math.log2(8))
```

3.4. Корни и степени

math.sqrt(x) — квадратный корень числа x .

math.pow(x, y) — возведение числа x в степень y .

math.expm1(x) — эквивалентно $\exp(x) - 1$, но более точный для малых x .

Пример:

```
import math
x = 16
y = 3
print("√16 =", math.sqrt(x))
print("2^3 =", math.pow(2, y))
print("expm1(0.0001) =", math.expm1(0.0001))
```

3.5. Округление и модуль

math.ceil(x) — округление числа x вверх до ближайшего целого.

math.floor(x) — округление числа x вниз до ближайшего целого.

math.trunc(x) — отбрасывание дробной части числа x .

math.fabs(x) — модуль числа x .

Пример:

```
import math
x = 3.7
print("ceil(3.7) =", math.ceil(x))
print("floor(3.7) =", math.floor(x))
print("trunc(3.7) =", math.trunc(x))
print("| -3.7 | =", math.fabs(-3.7))
```

3.6. Факториал и комбинаторика

math.factorial(x) — факториал числа x .

math.comb(n, k) — количество сочетаний из n по k .

math.perm(n, k) — количество размещений из n по k .

Пример:

```
import math
n = 5
k = 2
print("5! =", math.factorial(n))
print("C(5, 2) =", math.comb(n, k))
print("P(5, 2) =", math.perm(n, k))
```

3.7. Гамма-функция и связанные функции

math.gamma(x) — гамма-функция.

math.lgamma(x) — натуральный логарифм модуля гамма-функции.

Пример:

```
import math
x = 5.0
print("Г(5) =", math.gamma(x))
print("ln|Г(5)| =", math.lgamma(x))
```

4. Работа с углами

math.degrees(x) — преобразует угол x из радиан в градусы.

math.radians(x) — преобразует угол x из градусов в радианы.

Пример:

```
import math
angle_rad = math.pi / 2
angle_deg = 90
print("π/2 радиан =", math.degrees(angle_rad), "градусов")
print("90 градусов =", math.radians(angle_deg), "радиан")
```

5. Работа с бесконечностями и NaN

math.isinf(x) — проверяет, является ли x бесконечностью.

math.isnan(x) — проверяет, является ли x NaN (Not a Number).

Пример:

```
import math
x = float('inf')
y = float('nan')
print("x is infinite:", math.isinf(x))
print("y is NaN:", math.isnan(y))
```

6. Примеры использования модуля math

Рассмотрим несколько примеров, где модуль math может быть полезен.

Пример 1: Вычисление площади круга

```
import math
radius = 5
area = math.pi * math.pow(radius, 2)
print("Площадь круга с радиусом 5 =", area)
```

Пример 2: Решение квадратного уравнения

```
import math
a = 1
b = -3
c = 2
discriminant = math.pow(b, 2) - 4 * a * c
if discriminant > 0:
    x1 = (-b + math.sqrt(discriminant)) / (2 * a)
    x2 = (-b - math.sqrt(discriminant)) / (2 * a)
    print("Корни уравнения:", x1, x2)
elif discriminant == 0:
    x = -b / (2 * a)
    print("Корень уравнения:", x)
else:
    print("Корней нет")
```

Пример 3: Вычисление гипотенузы прямоугольного треугольника

```
import math
a = 3
b = 4
hypotenuse = math.hypot(a, b)
print("Гипотенуза треугольника с катетами 3 и 4 =", hypotenuse)
```

Модуль subprocess в Python

```
#Модуль subprocess - для работы с системными процессами, с их запуском и т.д.
import subprocess
exit = subprocess.call(['notepad.exe,']) #1 - название что запустить 2 - название файла
if exit == 0:
    print('good')
#можем запустить любой exe файл
subprocess.call([r'C:\Users\I.user\AppData\Local\Programs\Opera\launcher.exe', 'yandex.ru'])
#можно открывать и word файлы и т.д. в том числе и запуск питоновский исполняемых файлов
subprocess.call(['python.exe',
r'C:\Users\I.user\PycharmProjects\pythonProject1.py'])
#открытие питоновских файлов
#открытие процесса
#subprocess.Popen() Не ожидает завершения процесса
subprocess.Popen('notepad.exe')
subprocess.Popen('notepad.exe')
subprocess.Popen('notepad.exe').wait() #заставляет ожидать закрытия процесса
process = subprocess.Popen('notepad.exe')
process.wait(5) #ожидает закрытия процесса
```

Модуль threading в Python

Модуль threading в Python предоставляет возможность работы с потоками (threads), что позволяет выполнять несколько задач одновременно в рамках одного процесса. Это особенно полезно для задач, которые могут быть разделены на независимые части, такие как обработка данных, выполнение сетевых запросов или взаимодействие с пользовательским интерфейсом.

1. Импорт модуля threading

Прежде чем использовать функции и классы из модуля threading, его необходимо импортировать:

```
import threading
```

2. Основные концепции

2.1. Поток (Thread)

Поток – это отдельный поток выполнения в программе. Каждый поток имеет свой собственный стек вызовов, но разделяет память с другими потоками в том же процессе. Это позволяет потокам работать с общими данными, но также требует осторожности при работе с разделяемыми ресурсами, чтобы избежать состояний гонки (race conditions).

2.2. Глобальная блокировка интерпретатора (GIL)

В Python существует Global Interpreter Lock (GIL), который предотвращает выполнение нескольких потоков Python одновременно на уровне байт-кода. Это означает, что даже если у вас несколько потоков, только один из них может выполнять Python-код в любой момент времени.

Однако GIL не блокирует операции ввода-вывода (I/O), поэтому потоки могут быть полезны для задач, связанных с I/O.

3. Создание и запуск потоков

3.1. Создание потока с помощью функции

Поток можно создать, передав функцию в конструктор `threading.Thread`.

Пример:

```
import threading
import time

def worker():
    print("Поток запущен")
    time.sleep(2)
    print("Поток завершен")

Создание потока
thread = threading.Thread(target=worker)

Запуск потока
thread.start()

Ожидание завершения потока
thread.join()

print("Основной поток завершен")
```

3.2. Создание потока с помощью класса

Вы можете создать собственный класс, наследуя `threading.Thread`, и переопределить метод `run()`.

Пример:

```
import threading
import time

class MyThread(threading.Thread):
    def run(self):
        print("Поток запущен")
        time.sleep(2)
        print("Поток завершен")

Создание и запуск потока
thread = MyThread()
thread.start()
```

Ожидание завершения потока

```
thread.join()
```

```
print("Основной поток завершен")
```

4. Синхронизация потоков

4.1. Блокировки (Locks)

Блокировки используются для синхронизации доступа к разделяемым ресурсам. Они предотвращают одновременное выполнение критических участков кода несколькими потоками.

Пример:

```
import threading
```

Общий ресурс

```
counter = 0
```

```
lock = threading.Lock()
```

```
def increment():
```

```
    global counter
```

```
    for _ in range(100000):
```

```
        with lock:
```

```
            counter += 1
```

Создание потоков

```
threads = []
```

```
for i in range(10):
```

```
    thread = threading.Thread(target=increment)
```

```
    threads.append(thread)
```

```
    thread.start()
```

Ожидание завершения всех потоков

```
for thread in threads:
```

```
    thread.join()
```

```
print("Значение counter:", counter)
```

4.2. Условные переменные (Condition Variables)

Условные переменные используются для синхронизации потоков на основе определенных условий.

Пример:

```
import threading
```

Общий ресурс

```
items = []
```

```

condition = threading.Condition()

def producer():
    for i in range(5):
        with condition:
            items.append(i)
            print("Произведен элемент:", i)
            condition.notify() # Уведомление
            threading.Event().wait(1)

def consumer():
    while True:
        with condition:
            if not items:
                condition.wait() # Ожидание уведомления
            item = items.pop(0)
            print("Потреблен элемент:", item)
        if item == 4:
            break

```

Создание и запуск потоков

```

producer_thread = threading.Thread(target=producer)
consumer_thread = threading.Thread(target=consumer)

producer_thread.start()
consumer_thread.start()

producer_thread.join()
consumer_thread.join()

print("Основной поток завершен")

```

4.3. Семафоры (Semaphores)

Семафоры используются для ограничения количества потоков, которые могут одновременно получить доступ к ресурсу.

Пример:

```

import threading
import time

```

Семафор с ограничением в 2 потока

```

semaphore = threading.Semaphore(2)

def worker(id):

```


with semaphore:

```
print(f"Поток {id} получил доступ")
time.sleep(2)
print(f"Поток {id} освободил доступ")
```

Создание и запуск потоков

```
threads = []
for i in range(5):
    thread = threading.Thread(target=worker, args=(i,))
    threads.append(thread)
    thread.start()
```

Ожидание завершения всех потоков

```
for thread in threads:
    thread.join()

print("Основной поток завершен")
```

5. Примеры использования модуля threading

Рассмотрим несколько примеров, где модуль threading может быть полезен.

Пример 1: Параллельное выполнение задач

```
import threading
import time

def task(name, delay):
    print(f"Задача {name} начата")
    time.sleep(delay)
    print(f"Задача {name} завершена")

thread1 = threading.Thread(target=task, args=("A", 2))
thread2 = threading.Thread(target=task, args=("B", 1))
```

```
thread1.start()
thread2.start()
```

Ожидание завершения потоков

```
thread1.join()
thread2.join()

print("Основной поток завершен")
Пример 2: Параллельная обработка данных
python
```

Copy

```
import threading
```

Общий ресурс

```
data = [1, 2, 3, 4, 5]
```

```
result = []
```

```
lock = threading.Lock()
```

```
def process_data(item):
```

```
    with lock:
```

```
        result.append(item * 2)
```

Создание и запуск потоков

```
threads = []
```

```
for item in data:
```

```
    thread = threading.Thread(target=process_data, args=(item,))
```

```
    threads.append(thread)
```

```
    thread.start()
```

Ожидание завершения всех потоков

```
for thread in threads:
```

```
    thread.join()
```

```
print("Результат обработки данных:", result)
```

Пример 2: Пул потоков

```
import threading
```

```
import queue
```

```
import time
```

Очередь задач

```
task_queue = queue.Queue()
```

```
def worker():
```

```
    while True:
```

```
        task = task_queue.get()
```

```
        if task is None:
```

```
            break
```

```
        print(f"Обработка задачи: {task}")
```

```
        time.sleep(1)
```

```
        task_queue.task_done()
```

Создание и запуск потоков

```
num_worker_threads = 3
```

```
threads = []
for i in range(num_worker_threads):
    thread = threading.Thread(target=worker)
    thread.start()
    threads.append(thread)
```

Добавление задач в очередь

```
for item in range(10):
    task_queue.put(item)
```

Ожидание завершения всех задач

```
task_queue.join()
```

Остановка потоков

```
for i in range(num_worker_threads):
    task_queue.put(None)
for thread in threads:
    thread.join()

print("Основной поток завершен")
```

Создание и использование собственных модулей в Python

В Python модуль — это файл с расширением .py, содержащий код на Python. Модули позволяют организовывать код в логические блоки, что делает его более читаемым, поддерживаемым и повторно используемым. В этой лекции мы рассмотрим, как создавать собственные модули, импортировать их и использовать в других программах.

1. Что такое модуль?

Модуль — это файл Python, который может содержать функции, классы, переменные и другие объекты. Модули позволяют разбивать код на логические части, что упрощает его организацию и повторное использование.

Пример модуля:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

2. Создание модуля

Чтобы создать модуль, достаточно создать файл с расширением .py и написать в нем код. Например, создадим файл mymodule.py с двумя функциями:

```
# mymodule.py
```

```
def greet(name):  
    return f"Hello, {name}!"  
  
def add(a, b):  
    return a + b
```

Теперь этот файл можно использовать как модуль в других программах.

3. Импорт модуля

Чтобы использовать функции, классы или переменные из модуля, его нужно импортировать. Есть несколько способов импорта модулей.

3.1. Импорт всего модуля

Вы можете импортировать весь модуль и использовать его функции через точку.

Пример:

```
import mymodule  
  
print(mymodule.greet("Alice"))  
print(mymodule.add(2, 3))
```

3.2. Импорт конкретных функций

Вы можете импортировать только нужные функции из модуля.

Пример:

```
from mymodule import greet, add  
  
print(greet("Bob"))  
print(add(5, 7))
```

3.3. Импорт с псевдонимом

Вы можете импортировать модуль или функцию с псевдонимом (*alias*), чтобы использовать более короткое имя.

Пример:

```
import mymodule as mm  
  
print(mm.greet("Charlie"))  
print(mm.add(10, 20))
```

Или для конкретной функции:

```
from mymodule import greet as g  
  
print(g("Polina"))
```

4. Использование модулей в пакетах

Пакет — это способ организации нескольких модулей в одной директории. Пакет должен содержать файл `__init__.py` (может быть пустым), чтобы Python распознал его как пакет.

Пример структуры пакета:

```
mypackage/  
  __init__.py  
  module1.py  
  module2.py
```

4.1. Создание пакета

Создадим пакет mypackage с двумя модулями:

```
# mypackage/module1.py  
def func1():  
    return "Function 1"  
  
# mypackage/module2.py  
def func2():  
    return "Function 2"
```

Файл __init__.py может быть пустым или содержать код для инициализации пакета.

4.2. Импорт модулей из пакета

Вы можете импортировать модули из пакета следующим образом:

```
from mypackage import module1, module2  
  
print(module1.func1())  
print(module2.func2())
```

Или импортировать конкретные функции:

```
from mypackage.module1 import func1  
from mypackage.module2 import func2  
  
print(func1())  
print(func2())
```

5. Примеры использования собственных модулей

Пример 1: Простой модуль

Создадим модуль math_operations.py:

```
# math_operations.py  
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b
```

```
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Использование модуля:

```
import math_operations as mo

print(mo.add(10, 5))
print(mo.subtract(10, 5))
print(mo.multiply(10, 5))
print(mo.divide(10, 5))
```

Пример 2: Пакет с несколькими модулями

Создадим пакет **shapes** с модулями **circle.py** и **rectangle.py**:

```
# shapes/circle.py
import math

def area(radius):
    return math.pi * radius ** 2

def circumference(radius):
    return 2 * math.pi * radius

# shapes/rectangle.py
def area(length, width):
    return length * width

def perimeter(length, width):
    return 2 * (length + width)
```

Использование пакета:

```
from shapes.circle import area as circle_area, circumference
from shapes.rectangle import area as rectangle_area, perimeter

print("Circle Area:", circle_area(5))
print("Circle Circumference:", circumference(5))
print("Rectangle Area:", rectangle_area(4, 6))
print("Rectangle Perimeter:", perimeter(4, 6))
```

6. Управление видимостью объектов в модуле

В Python нет строгой инкапсуляции, но принято использовать префикс `_` для обозначения "защищенных" объектов и `__` для "приватных" объектов.

Пример:

```
# mymodule.py
def public_function():
    return "This is a public function"
def _protected_function():
    return "This is a protected function"
def __private_function():
    return "This is a private function"
```

При импорте модуля объекты с префиксом `_` не будут импортированы с помощью `from module import *`, но все равно доступны явно.

Генераторы и декораторы в Python

Генераторы и декораторы — это мощные инструменты в Python, которые позволяют писать более эффективный и выразительный код. Генераторы используются для создания итераторов, а декораторы — для изменения поведения функций или методов.

Часть 1: Генераторы

Генераторы — это специальные функции, которые позволяют лениво (lazy) генерировать последовательности значений. В отличие от обычных функций, которые возвращают результат и завершают выполнение, генераторы возвращают значения по одному с помощью ключевого слова `yield`. Это позволяет экономить память, так как значения генерируются "на лету", а не хранятся в памяти целиком.

1.1. Создание генератора

Генератор создается с помощью функции, которая использует ключевое слово `yield`. Когда функция встречает `yield`, она приостанавливает выполнение и возвращает значение. При следующем вызове функция продолжает выполнение с того места, где она остановилась.

Пример простого генератора:

```
def simple_generator():
    yield 1
    yield 2
    yield 3

# Использование генератора
gen = simple_generator()
print(next(gen)) # Вывод: 1
print(next(gen)) # Вывод: 2
print(next(gen)) # Вывод: 3
```

1.2. Генераторы и цикл for

Генераторы можно использовать в цикле `for`, так как они являются итераторами:

```
def countdown(n):
```

```
while n > 0:
    yield n
    n -= 1
```

```
# Использование генератора в цикле
for number in countdown(5):
    print(number)
```

Вывод:

```
5
4
3
2
1
```

1.3. Генераторные выражения

Генераторные выражения похожи на списковые включения (list comprehensions), но возвращают генератор вместо списка. Они создаются с использованием круглых скобок.

Пример:

```
# Генераторное выражение
squares = (x ** 2 for x in range(10))

# Использование генератора
for square in squares:
    print(square)
```

1.4. Преимущества генераторов

Экономия памяти: Генераторы не хранят всю последовательность в памяти, а генерируют значения по мере необходимости.

Ленивые вычисления: Значения вычисляются только тогда, когда они запрашиваются.

Бесконечные последовательности: Генераторы могут генерировать бесконечные последовательности.

Пример бесконечного генератора:

```
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1

# Использование бесконечного генератора
gen = infinite_sequence()
for _ in range(5):
    print(next(gen))
```


Часть 2: Декораторы

Декораторы — это функции, которые принимают другую функцию и расширяют или изменяют ее поведение, не изменяя ее исходный код. Декораторы часто используются для добавления логики, такой как логирование, кэширование, проверка прав доступа и т.д.

2.1. Создание декоратора

Декоратор — это функция, которая принимает функцию в качестве аргумента и возвращает новую функцию.

Пример простого декоратора:

```
def my_decorator(func):
    def wrapper():
        print("До выполнения функции")
        func()
        print("После выполнения функции")
    return wrapper

# Применение декоратора
@my_decorator
def say_hello():
    print("Привет!")

# Вызов функции
say_hello()
```

Вывод:

До выполнения функции

Привет!

После выполнения функции

2.2. Декораторы с аргументами

Если декорируемая функция принимает аргументы, декоратор должен использовать `*args` и `**kwargs` для передачи аргументов.

Пример:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("До выполнения функции")
        result = func(*args, **kwargs)
        print("После выполнения функции")
        return result
    return wrapper

@my_decorator
def add(a, b):
```

```
return a + b

print(add(2, 3))
```

Вывод:

До выполнения функции

После выполнения функции

5

2.3. Декораторы с параметрами

Иногда нужно передавать параметры самому декоратору. Для этого создается дополнительный уровень вложенности.

Пример:

```
def repeat(num_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Привет, {name}!")

greet("Алиса")
```

Вывод:

Привет, Алиса!

Привет, Алиса!

Привет, Алиса!

2.4. Встроенные декораторы

Python предоставляет несколько встроенных декораторов, таких как `@staticmethod`, `@classmethod` и `@property`.

Пример использования `@property`:

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
```

```
    return self._radius

@radius.setter
def radius(self, value):
    if value < 0:
        raise ValueError("Радиус не может быть отрицательным")
    self._radius = value

circle = Circle(5)
print(circle.radius) # Вывод: 5
circle.radius = 10
print(circle.radius) # Вывод: 10
```

2.5. Цепочка декораторов

Декораторы можно применять последовательно. Порядок применения декораторов важен: они выполняются снизу вверх.

Пример:

```
def decorator1(func):
    def wrapper():
        print("Декоратор 1")
        func()
    return wrapper

def decorator2(func):
    def wrapper():
        print("Декоратор 2")
        func()
    return wrapper

@decorator1
@decorator2
def say_hello():
    print("Привет!")

say_hello()
```

Вывод:

```
Декоратор 1
Декоратор 2
Привет!
```

Часть 3: Примеры использования

3.1. Генератор для чтения больших файлов

Генераторы полезны для обработки больших файлов, так как они не загружают весь файл в память.

Пример:

```
def read_large_file(file_path):  
    with open(file_path, "r") as file:  
        for line in file:  
            yield line.strip()  
  
# Использование генератора  
for line in read_large_file("large_file.txt"):  
    print(line)
```

3.2. Декоратор для измерения времени выполнения функции

Декораторы можно использовать для измерения времени выполнения функции.

Пример:

```
import time  
  
def timer(func):  
    def wrapper(*args, **kwargs):  
        start_time = time.time()  
        result = func(*args, **kwargs)  
        end_time = time.time()  
        print(f"Функция {func.__name__} выполнялась за {end_time - start_time:.4f} секунд")  
        return result  
    return wrapper  
  
@timer  
def slow_function():  
    time.sleep(2)  
  
slow_function()
```

Вывод:

```
Функция slow_function выполнялась за 2.0001 секунд
```

Обработка исключений в Python (try, except)

Обработка исключений — это механизм, который позволяет программе корректно реагировать на ошибки, возникающие во время выполнения. В Python для этого используются конструкции try, except, else, finally.

1. Что такое исключения?

Исключения (exceptions) — это ошибки, которые возникают во время выполнения программы. Они могут быть вызваны различными причинами, такими как:

Деление на ноль.

Обращение к несуществующему элементу списка.

Попытка открыть несуществующий файл.

Неправильный ввод пользователя.

Если исключение не обработано, программа завершается с ошибкой. Однако с помощью конструкции try-except можно "поймать" исключение и обработать его.

2. Базовый синтаксис try-except

Конструкция try-except позволяет "попробовать" выполнить код и обработать возможные ошибки.

2.1. Пример: Деление на ноль

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
```

Вывод:

Ошибка: деление на ноль!

2.2. Как это работает?

Код внутри блока try выполняется.

Если возникает исключение, выполнение блока try прерывается, и управление передается в блок except.

Если исключение не возникает, блок except пропускается.

3. Обработка нескольких исключений

Можно обрабатывать разные типы исключений с помощью нескольких блоков except.

3.1. Пример: Обработка разных ошибок

```
try:
    num = int(input("Введите число: "))
    result = 10 / num
    print("Результат:", result)
except ValueError:
    print("Ошибка: введено не число!")
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
```

Пример работы:

Введите число: abc

Ошибка: введено не число!

Введите число: 0

Ошибка: деление на ноль!

4. Блок else

Блок `else` выполняется, если в блоке `try` не возникло исключений. Он полезен для кода, который должен выполняться только при успешном завершении блока `try`.

4.1. Пример: Использование else

```
try:
    num = int(input("Введите число: "))
    result = 10 / num
except ValueError:
    print("Ошибка: введено не число!")
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
else:
    print("Результат:", result)
```

Пример работы:

Введите число: 2

Результат: 5.0

5. Блок finally

Блок `finally` выполняется всегда, независимо от того, возникло исключение или нет. Он используется для выполнения обязательных действий, таких как закрытие файлов или освобождение ресурсов.

5.1. Пример: Использование finally

```
try:
    file = open("example.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("Ошибка: файл не найден!")
finally:
    print("Закрытие файла")
    file.close()
```

Пример работы:

Ошибка: файл не найден!

Закрытие файла

6. Исключения с пользовательскими сообщениями

Можно выводить пользовательские сообщения об ошибках, используя объект исключения.

6.1. Пример: Использование объекта исключения

```
try:
    num = int(input("Введите число: "))
    result = 10 / num
except ValueError as e:
    print(f"Ошибка: {e}")
except ZeroDivisionError as e:
    print(f"Ошибка: {e}")
```

Пример работы:

Введите число: abc

Ошибка: invalid literal for int() with base 10: 'abc'

7. Пользовательские исключения

Вы можете создавать собственные исключения, наследуясь от класса Exception.

7.1. Пример: Создание пользовательского исключения

```
class NegativeNumberError(Exception):
    def __init__(self, message="Число не может быть отрицательным"):
        self.message = message
        super().__init__(self.message)

def check_positive(number):
    if number < 0:
        raise NegativeNumberError
    return number

try:
    num = int(input("Введите положительное число: "))
    check_positive(num)
except NegativeNumberError as e:
    print(e)
```

Пример работы:

Введите положительное число: -5

Число не может быть отрицательным

8. Примеры использования

8.1. Пример: Обработка ошибок при работе с файлами

```
try:
    with open("data.txt", "r") as file:
        content = file.read()
        print(content)
```

```
except FileNotFoundError:
    print("Файл не найден!")
except PermissionError:
    print("Нет прав доступа к файлу!")
except Exception as e:
    print(f"Произошла ошибка: {e}")
```

8.2. Пример: Обработка ошибок ввода пользователя

```
while True:
    try:
        age = int(input("Введите ваш возраст: "))
        if age < 0:
            raise ValueError("Возраст не может быть отрицательным")
        break
    except ValueError as e:
        print(f"Ошибка: {e}")

print(f"Ваш возраст: {age}")
```

9. Иерархия исключений

В Python все исключения наследуются от базового класса `BaseException`. Наиболее часто используемым классом является `Exception`. Вот некоторые стандартные исключения:

ValueError: Некорректное значение.

TypeError: Некорректный тип данных.

IndexError: Выход за границы списка.

KeyError: Ключ не найден в словаре.

FileNotFoundError: Файл не найден.

ZeroDivisionError: Деление на ноль.